uncertain-gpt: a shared-kernel benchmark suite for the training cost of Bayesian deep learning

Tage Sköldefors

August 2026

Abstract

uncertain-gpt is a benchmark suite for what a Bayesian deep learning method costs to train. One CUDA implementation of a GPT-2 of 11,061,531 parameters supplies the kernels, the forward and backward pass, the dataloader, the checkpoint format, the timing harness and the evaluation harness, and the AdamW constants, the batch order and fp32 arithmetic are shared as well, so an arm differs from every other arm only in what it stores and how many passes it makes over that store. A method enters as an update kernel and a callback returning posterior draw s. Bayes-by-Backprop, Vadam, a learned low-rank-plus-diagonal Gaussian covariance and a five-member deep ensemble are the reference arms, each a method from published work. Across a 256-fold batch sweep at fixed sequence length, each arm's posterior work holds to within 1.5% of itself while tokens per step go from 64 to 16,384, so its share of the step falls towards $1/B$ above $B \approx 16$ and more slowly below it, where the network's own passes are launch-bound rather than token-bound. Folding an arm's parameter-sized passes into one kernel cuts Bayes-by-Backprop's $O(P)$ time by 1.76× and leaves Vadam's inside the 1% band the unchanged control arm reproduces itself in, which Vadam's 11 buffer touches per step against a floor of 9 predicted before the sweep ran. Vadam's fitted posterior at this scale is the prior it started from, and an epistemic term is biased low until its sample count converges, at an S that differs by arm.

1 Introduction

A training step's cost is a property of an implementation as much as of a method. Two Bayesian methods timed in two codebases differ in their kernels, their data path, their optimiser constants and their precision, and a gap in step time between them is attributable to no one of those.

uncertain-gpt holds all of them fixed. One CUDA implementation of a GPT-2 [20] supplies the kernels, the forward and backward pass, the dataloader, the checkpoint format, the timing harness and the evaluation harness. A method enters as an update kernel and a callback that returns posterior draw s, and inherits the rest, so two arms differ only in what they store and how many passes they make over that store. Sharing kernels requires owning them: a framework supplies the optimiser, the allocator and the choice of kernel, which are the terms a shared implementation exists to remove, so the network is written from scratch in C and CUDA and every kernel outside cuBLAS is checked against a single-threaded C reference.

The four arms implemented against that shared code are methods from published work rather than stand-ins for them: Bayes-by-Backprop [1], Vadam [11], a learned low-rank-plus-diagonal Gaussian covariance, and a five-member deep ensemble [14] as the control, which stores no posterior and therefore prices what the other three add.

Section 5 reports four measurements on them. Two are readings a count of an arm's buffers implies and the suite confirms: a posterior's per-step cost is flat to within 1.5% across a 256-fold

batch sweep, and folding an arm's parameter-sized passes into one kernel returns time in proportion to the traffic it deletes, which for Vadam is nothing. Two are properties of the posteriors themselves: Vadam's fitted posterior at this scale is the prior it started from, and an epistemic term is biased low until its sample count converges, at an S that differs by arm.

2 Related work

Weight uncertainty was priced as description length by Hinton and van Camp [7], made practical for real networks by Graves [5], and turned into Bayes-by-Backprop by Blundell et al. [1] through the reparameterisation of Kingma and Welling [13], which pays for a posterior with a second parameter-sized tensor and a second optimiser state; Wen et al. [25] move a per-example perturbation into the activations. Khan et al. [11] instead reuse Adam's second moment as a posterior precision, Osawa et al. [18] scale natural-gradient variational inference to ImageNet, and Shen et al. [23] apply it at GPT-2 scale. Covariances fitted after training move the cost out of the step: a Kronecker-factored Laplace approximation at a trained point [21], the same approximation as a recipe [3], and a low-rank-plus-diagonal factor from the SGD trajectory [16], which is the covariance the third arm here learns during training instead. Deep ensembles [14] keep the step deterministic and pay E training runs, and match or beat variational methods on the benchmarks of Ovadia et al. [19]; MC dropout [4] is the other cheap baseline, and this network has no dropout layers. Metrics follow Guo et al. [6] and Kendall and Gal [10].

3 The suite

What every arm shares. A GPT-2 [20] of the usual pre-LayerNorm shape [24] at $C = 384$, 6 heads, 6 blocks, $V = 27$, trained at $T = 64$ and $B = 4$: 11,061,531 parameters, the six blocks holding about 96%. Attention is the IO-aware formulation of Dao et al. [2]; format, loader and reference implementation follow llm.c [9] and nanoGPT [8]. The corpus is text8 [17], character-level over 27 symbols in the standard 90M/5M/5M split, walked sequentially so every arm sees the same tokens in the same order, for 100,000 steps. One RTX 2060 Max-Q, `sm_75`, 6 GB, fp32 throughout, the four GEMMs per block through cuBLAS. AdamW [12, 15] at learning rate 10^{-4}, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, no weight decay, no schedule, and Philox 4x32-10 [22] for randomness, so a perturbation is a pure function of (seed, step, weight index), regenerated in the backward pass rather than stored.

What an arm supplies. Two things: a kernel that updates whatever the method stores, and a `uq_sample_fn`, a callback returning a device pointer to posterior draw s. Roughly 150 lines of CUDA in the reference arms, against a shared header of 1,610. Nothing else is an arm's to change. The buffer layout is one table both the network and every arm index through, and an arm's file is compiled both as its own binary and into a Python extension, so the kernels a reader times and the kernels a reader imports are the same kernels. Section 8 states the three conventions that requires.

Timing a region. Step times come from the real training step with CUDA event marks at region boundaries, one region per kernel group, rather than from a copy of it that can drift, and a region's name records whether its cost is $O(P)$ and Bayesian, $O(P)$ and paid by a deterministic trainer too, or $O(B \cdot T)$. Warm-up is discarded and the statistic is a median with $p25$ and $p75$ beside it, so a throttled bimodal sample stays visible instead of smearing into a mean. Temperature and SM clock are probed every 20 steps and printed with each measurement, the binary aborts above 89 °C, and the driver idles between points until the card is under 72 °C. A point with less than 384 MB

Region	Touches	ms at $B = 4$	GB/s	ms at $B = 128$	GB/s
`*sample`	3	0.5644	235.2	0.5655	234.7
`+zero_dw`	1	0.1744	253.7	0.1745	253.6
`*zero_post`	2	0.3500	252.8	0.3502	252.7
`*post_bwd`	6	1.0967	242.1	1.0858	244.5
`*kl`	6	1.0984	241.7	1.0940	242.7
`+adamw_mu`	7	1.2977	238.7	1.2914	239.8
`*adamw_rho`	7	1.2972	238.8	1.2923	239.7
all $O(P)$ work	32	5.8788	240.8	5.8537	241.9
`*fused_step`	15	3.4410	192.9	3.5422	187.4

Table 1: Every parameter-sized region of the mean-field step, touches counted at the launch sites and converted at $P \cdot 4$ bytes each. The aggregate row is the sum of the rows above it; `*fused_step` is the same work after folding.

of VRAM headroom is reported out of range rather than measured (Section 7). The sweep runs $B = 1$ to 256 at $T = 64$ held. Touch counts, one read or one write of a parameter-sized buffer, are cross-checked rather than trusted: converting a region's time to achieved bandwidth (Table 1) has to put structurally unrelated regions in one band.

Measuring quality. Evaluation loads a checkpoint and never trains, and each arm supplies only the callback returning draw s. Over S draws per position, with predictive distributions p_s and mixture $\bar{p} = \frac{1}{S}\sum_s p_s$, the shared harness reports negative log predictive density of mixture and single draw, expected calibration error over 15 bins of argmax confidence [6], and the decomposition [10]

$$\underbrace{H[\bar{p}]}_{\text{total}} = \underbrace{\tfrac{1}{S}\textstyle\sum_s H[p_s]}_{\text{aleatoric}} + \underbrace{\tfrac{1}{S}\textstyle\sum_s \mathrm{KL}(p_s \,\|\, \bar{p})}_{\text{epistemic}}, \tag{1}$$

in nats, in double precision, mixing samples as probabilities and never as logits. The off-distribution stream is uniform-random token ids from a fixed seed. S is the first value at which every arm free to choose one has converged, $S = 64$ for these four; an arm that converges later raises it rather than reporting below its own converged value.

What can be proved about a change to the suite. Three instruments, in increasing strength: an evaluation whose output is byte-comparable across a change to the training path, a kernel-equivalence harness that can demand exact bit equality by rebuilding with multiply-add contraction disabled, and a trajectory diff against source rebuilt from an earlier commit. Section 6 reports what they returned here.

4 Reference implementations

Four methods are implemented against the shared code of Section 3, each counted twice below: the passes the method needs, and the passes left after the redundant ones are folded into one kernel. At $P = 11{,}061{,}531$ parameters in fp32 a touch is 44.2 MB, and a deterministic AdamW step costs eight: a gradient zeroed and read, weights and two moments each read and written. One observation does most of the folding in all three Bayesian arms. A gradient accumulator for a variational parameter is write-only, so it becomes a register rather than reaching memory, and the next step's draw folds in with it as a function of the parameters the update just wrote.

Bayes-by-Backprop. A fully factorised Gaussian, drawn as $w_i = \mu_i + \mathrm{softplus}(\rho_i)\,\varepsilon_i$ once per step for the whole batch and reparameterised so the gradient reaches both moments [13, 1], against a prior of standard deviation 0.1 under an ELBO whose KL weight is $1/N$ for N the training token count, read off the loader rather than tuned. It stores μ, ρ and an AdamW moment pair for each, and runs AdamW twice a step: eight launches and 32 touches over ten buffers. Folding leaves one launch and 15 touches over eight, the floor its state implies.

Vadam. Adam's second moment already estimates a curvature, so Khan et al. [11] reuse it as the posterior precision and allocate nothing for the posterior. The scale is read off the optimiser,

$$\sigma_i = \frac{1}{\sqrt{N s_i + \lambda}}, \qquad w_i = \mu_i + \sigma_i \varepsilon_i, \tag{2}$$

with $\lambda = 1/\sigma_{\text{init}}^2 = 2500$ the prior precision, the initialisation rather than a tuned value, and the square root absent from σ following that paper's footnote 2. Its update is Figure 1 of the same paper transcribed unchanged, one Adam step with the prior folded into the gradient, so the state is μ with Adam's own m and s: three launches and 11 touches against a floor of 9, two touches of slack and no accumulator to delete.

Low-rank plus diagonal covariance. This arm buys correlation with memory. The covariance is $\Sigma = AA^\top + \mathrm{diag}(d^2)$ at rank $r = 8$, never formed, and a draw is $w_i = \mu_i + \sum_{k<r} A_{ik} z_k + d_i \varepsilon_i$ with one $z \sim \mathcal{N}(0, I_r)$ per weight matrix per forward pass, shared by all of that matrix's weights, which is the coupling; the KL reaches $\log\det\Sigma$ through an $r \times r$ determinant. The posterior covers the four GEMM weight matrices per block, 24 tensors holding 10,616,832 parameters. Counted per Bayesian layer in units of that layer's parameter count, so that A counts as r, it costs two launches per layer plus four global, 122 units at $r = 8$, folding to one launch per layer and 58. It is the one arm left above its floor: its KL setup goes through a cuBLAS call per layer and a host Cholesky, so it cannot fold into the update it feeds.

Deep ensemble (control). $E = 5$ deterministic models [14] differing only in the initialisation seed, which with no dropout and a sequential loader is the only source of diversity; member probabilities are averaged at inference, never logits. The arm stores no posterior and introduces no kernels, so its step is the deterministic two launches and eight touches, already at the floor. It is left unfused on purpose: its code is then identical in both sweeps, which makes it the drift control in Section 5. Its cost sits elsewhere, in E times the step, E times the memory, E training runs, and an evaluation sample count capped at the members it trained.

5 What the suite shows

5.1 A posterior's per-step cost does not contain the batch size

Each arm's posterior work costs the same milliseconds at 64 tokens per step as at 16,384: across the 256-fold sweep the directly timed Bayesian regions hold to within 1.5% of themselves on every arm (Table 2), the width of the measurement and not a trend in it. Nothing in those regions has a token in it.

The network's cost grows with tokens and the posterior's does not, so the posterior's share of a step falls, at a local exponent of 1.005 for mean-field over the last two points. That asymptote is not the whole curve: between $B = 1$ and $B = 2$ the exponent is 0.148 for mean-field and 0.196 for

Arm	B range	Posterior ms (min)	(max)	Spread	Share at $B = 1$	Share at max B	k (small end)	k (top end)
BBB	1–256	4.390	4.437	1.1%	34.40%	0.75%	0.148	1.005
Vadam	1–256	1.844	1.872	1.5%	20.52%	0.31%	0.196	0.998
LowR	1–128	38.470	39.001	1.4%	79.71%	11.65%	0.036	0.824

Table 2: Each arm's own Bayesian work, timed at the launch sites, unfused. Share is the posterior over the whole step, k the exponent in share $\propto B^{-k}$ across the two smallest and the two largest points.

B	Tokens/step	BBB ms	BBB $O(P)$%	Vadam ms	Vadam $O(P)$%	LowR ms	LowR $O(P)$%	Ens ms	Ens $O(P)$%
1	64	10.71	31.4	9.16	21.6	42.66	80.8	8.45	17.7
2	128	11.99	28.2	10.42	19.1	44.20	78.7	9.83	15.1
4	256	15.23	22.6	13.77	14.7	47.49	74.1	13.18	11.2
8	512	24.18	14.3	22.69	9.0	56.99	62.2	22.16	6.7
16	1024	43.18	8.1	41.68	4.9	76.04	46.4	41.20	3.6
32	2048	79.27	4.4	77.78	2.6	113.13	31.4	77.70	1.9
64	4096	149.81	2.3	148.59	1.4	182.67	19.2	148.82	1.0
128	8192	295.16	1.2	294.54	0.7	329.50	10.7	292.52	0.5

Table 3: The batch sweep for the shipped code. $O(P)$% is all parameter-sized work over the step, Bayesian or not; the ensemble column is one member.

Vadam, because the network is not $O(B \cdot T)$ at the bottom of the sweep either, twice the tokens costing the control arm a factor of 1.16 there and a factor of two from $B = 16$ up (Table 3). Nor is the fixed numerator a property of posteriors: the control's own parameter-sized share falls just as far.

What fails to amortise is state not proportional to the parameter count, which is why the low-rank arm still owns 11.65% of a step at its largest measurable batch where mean-field owns 0.75% and Vadam 0.31%. All of it is per-step cost, pricing neither the ensemble's training runs nor a variational arm's evaluation passes.

5.2 Folding returns the redundant traffic and nothing else

Folding an arm's parameter-sized passes into one kernel buys time in proportion to the traffic it deletes. Mean-field's 32 touches drop to 15 for 1.76× of $O(P)$ time at $B = 1$, the low-rank arm's 122 per layer to 58 for 1.14 to 1.16×, and Vadam's 11 to 9 for between 0.98 and 1.04× (Table 4). The control's code is unchanged in both sweeps and comes out within 1% of itself, so that band is drift, and Vadam's result sits inside it.

Vadam's pass count predicted that null before the sweep ran. Two touches of slack against a floor of 9 is 1.22× of traffic, and a fused kernel gives back bandwidth: mean-field's streams eight buffers at once where the sequence it replaced walked three or four each (Table 1). At that discount a 2.13× traffic cut is still 1.76× of time and a 1.22× cut is nothing.

What folding returns is a fixed count of buffer touches, so it amortises exactly as the posterior's own cost does: the same 1.76× is a fifth of a mean-field step at $B = 1$ and under a percent of one at $B = 128$ (Table 5). Taking the boundary per arm at the 1% the control reproduces itself in, mean-field crosses at $B = 128$, Vadam is inside that band throughout, and the low-rank arm is outside it wherever this card can run.

Arm	B	$O(P)$ ms before	$O(P)$ ms after	×	ms/step before	ms/step after	$O(P)$ % before	$O(P)$ % after	share ratio
BBB	1	5.92	3.36	1.76	12.90	10.71	45.9	31.4	0.68
BBB	2	5.90	3.38	1.74	14.25	11.99	41.4	28.2	0.68
BBB	4	5.88	3.44	1.71	17.48	15.23	33.7	22.6	0.67
BBB	8	5.87	3.46	1.70	26.36	24.18	22.3	14.3	0.64
BBB	16	5.86	3.48	1.68	45.07	43.18	13.0	8.1	0.62
BBB	32	5.86	3.50	1.67	81.03	79.27	7.2	4.4	0.61
BBB	64	5.86	3.48	1.68	150.46	149.81	3.9	2.3	0.60
BBB	128	5.86	3.54	1.65	293.16	295.16	2.0	1.2	0.60
Vadam	1	2.05	1.98	1.04	9.12	9.16	22.4	21.6	0.96
Vadam	2	2.05	1.99	1.03	10.44	10.42	19.6	19.1	0.98
Vadam	4	2.03	2.03	1.00	13.73	13.77	14.8	14.7	0.99
Vadam	8	2.03	2.04	1.00	22.61	22.69	9.0	9.0	1.00
Vadam	16	2.03	2.05	0.99	41.40	41.68	4.9	4.9	1.01
Vadam	32	2.02	2.06	0.98	76.83	77.78	2.6	2.6	1.01
Vadam	64	2.03	2.05	0.99	148.36	148.59	1.4	1.4	1.01
Vadam	128	2.02	2.07	0.98	293.82	294.54	0.7	0.7	1.02
LowR	1	39.94	34.48	1.16	48.26	42.66	82.7	80.8	0.98
LowR	2	40.03	34.77	1.15	49.61	44.20	80.7	78.7	0.97
LowR	4	40.34	35.18	1.15	52.84	47.49	76.3	74.1	0.97
LowR	8	40.47	35.45	1.14	62.11	56.99	65.2	62.2	0.95
LowR	16	40.37	35.28	1.14	81.06	76.04	49.8	46.4	0.93
LowR	32	40.42	35.57	1.14	117.51	113.13	34.4	31.4	0.91
LowR	64	40.11	35.06	1.14	187.41	182.67	21.4	19.2	0.90
LowR	128	40.43	35.21	1.15	334.62	329.50	12.1	10.7	0.88
Ens	1	1.49	1.50	1.00	8.56	8.45	17.4	17.7	1.01
Ens	2	1.49	1.49	1.00	9.95	9.83	14.9	15.1	1.01
Ens	4	1.49	1.48	1.00	13.33	13.18	11.1	11.2	1.01
Ens	8	1.48	1.48	1.00	22.37	22.16	6.6	6.7	1.01
Ens	16	1.47	1.47	1.00	41.50	41.20	3.6	3.6	1.01
Ens	32	1.48	1.47	1.00	78.24	77.70	1.9	1.9	1.00
Ens	64	1.47	1.47	1.00	148.24	148.82	1.0	1.0	1.00
Ens	128	1.47	1.48	0.99	295.64	292.52	0.5	0.5	1.02

Table 4: Parameter-sized work before and after folding, where both sweeps measured the point. The ensemble's code is identical in both, so its rows are the drift.

5.3 Vadam's fitted posterior is its prior

What the arms return at a fixed budget of 100,000 steps is read from their checkpoints, on both splits with neither preferred. The ensemble has the best mixture NLPD and the worst ECE by close to a factor of two and Vadam holds the opposite corner (Table 6), with mean-field and the low-rank arm between them, no ranking being claimed on the NLPD column: with one run per arm, nothing here measures the noise floor its spread would have to clear.

A larger epistemic term is not a better posterior. Vadam's in-distribution epistemic is 0.0381 nats on validation against mean-field's 0.0108 (Table 7), more than three times it, while its separation between in- and off-distribution input sits with the other variational arms at 1.197 (Table 8). The spread is the prior's. Equation 2 weighs the data's contribution to the precision, Ns, against $\lambda = 2500$, so the per-token RMS gradient would have to reach $\sqrt{\lambda/N} = 5.27 \times 10^{-3}$ where the median observed value is 2.71×10^{-5}, short by 195×.[1] At that median the data supplies 0.0026% of the precision and 99.8% of parameters sit within 10% of the prior standard deviation; the minibatch bias of that paper's own Theorem 1 accounts for at most 16× of the shortfall on the gradient scale, and correcting for it in full moves that 99.8% to 85.1%.

The mean did fit: μ ends at correlation 0.083 with its initialisation, having moved 1.27 times

[1] Read from the trained checkpoint by `dev/vadam_posterior.py`, the one measurement here taken from a checkpoint rather than a logged run.

Arm	Folding returns (ms, min)	(max)	% of step at $B = 1$	at $B = 128$	Posterior % at $B = 128$	B^*
BBB	2.320	2.561	19.85%	0.79%	1.50%	128
Vadam	-0.049	0.070	0.76%	-0.02%	0.63%	≤ 1
LowR	4.851	5.458	11.31%	1.56%	11.65%	> 128

Table 5: Where folding stops being separable from drift. What it returns is $O(P)$ time deleted, over the unfused step; B^* is the first batch size measured at which that lands inside the control's band, or a bound where the sweep never brackets it.

Arm	Split	S	NLPD (single draw)	NLPD (mixture)	ECE
BBB	val	64	1.5642	1.5585	0.0246
BBB	test	64	1.6611	1.6513	0.0168
Vadam	val	64	1.7119	1.6768	0.0176
Vadam	test	64	1.7748	1.7411	0.0135
LowR	val	64	1.5982	1.5816	0.0206
LowR	test	64	1.6806	1.6636	0.0189
Ens	val	5	1.5255	1.4708	0.0490
Ens	test	5	1.6251	1.5672	0.0379

Table 6: Predictive quality, in nats, over 16,384 positions. "Single draw" is one posterior sample, or member 0 for the ensemble: comparable within an arm, not across them.

its own initial norm. What the width reports is the threshold $\lambda = 1/\sigma^2_{\text{init}}$ sets against this model's gradients, not a run that failed to train, and noise that does not shrink as the mean fits is why that arm fails the one-batch memorisation gate.

5.4 An epistemic term is a Monte Carlo estimate at some S

Every uncertainty number above carries its S: the estimator averages over S draws, is exactly zero at $S = 1$ by construction, one sample disagreeing with nothing, and approaches its converged value from below (Figure 1). Where it converges is a property of the arm rather than of the harness: mean-field is flat by $S = 32$ and confirmed flat to $S = 128$, though its ECE rises at every doubling over that range while its mixture NLPD improves. Vadam and the low-rank arm are still climbing at $S = 32$ and flatten only at $S = 64$, which is why Section 3's convention sits there rather than where mean-field alone would put it.

The ensemble cannot be read at an S of anyone's choosing. Its s-th sample is its s-th trained member, so S is capped at $E = 5$ and a further draw costs a training run rather than a forward pass. Tables 7 and 8 state it at $S = 5$ beside variational arms at $S = 64$, and every epistemic term compared across those rows carries that difference.

6 Verification

Folding is a claim about an implementation, so the numbers of Section 5.2 rest on gates a wrong fold fails. Evaluation at $S = 32$ on validation and test is byte-identical before and after on all four arms, and is the weakest of the three: it draws with the unfused sample kernels, none of which folding touches, so a training path could be arbitrarily wrong and this would still pass. What it establishes is that the inference path and the checkpoints were not disturbed. Running each fused

Arm	Split	S	Total	Aleatoric	Epistemic (ID)
BBB	val	64	1.602	1.591	0.0108
BBB	test	64	1.660	1.649	0.0104
Vadam	val	64	1.708	1.670	0.0381
Vadam	test	64	1.740	1.703	0.0364
LowR	val	64	1.620	1.603	0.0171
LowR	test	64	1.673	1.656	0.0169
Ens	val	5	1.628	1.561	0.0672
Ens	test	5	1.690	1.622	0.0674

Table 7: In-distribution decomposition, in nats, by Equation 1. The worst residual against the asserted identity behind any number here is 4.4×10^{-15}.

Arm	S	Epistemic (OOD)	OOD/ID (val)	OOD/ID (test)
BBB	64	0.0120	1.111	1.154
Vadam	64	0.0456	1.197	1.253
LowR	64	0.0216	1.263	1.278
Ens	5	0.1733	2.579	2.571

Table 8: Epistemic uncertainty on the off-distribution stream, in nats, stated once per arm: the stream is one fixed synthetic sequence, so only the ratio's denominator is split-dependent.

kernel and the kernels it replaces on identical inputs leaves mean-field and Vadam bit-identical on every output buffer and the low-rank arm within 1.00 ULP, scored at the larger of a value's own magnitude and the buffer's RMS; rebuilding with multiply-add contraction disabled makes all three bit-identical, which puts the residual in contraction rather than in the arithmetic. And 100 steps of normal-mode training against the pre-folding source reproduce every printed step line on all four arms, including both summary lines of the control, whose in-training output is the reference the repository is calibrated against.

The identity of Equation 1 is asserted at 10^{-5} in double precision on every evaluation and the process exits nonzero on a violation; the worst residual behind any number in this paper is 4.4×10^{-15}. It certifies less than that suggests. Both halves of the right-hand side are computed directly rather than one from the other, so a violation is an error in the accumulation. But the identity is normalisation-free, since $\bar{p}$ is the mean of the very vectors it is checked against, and it therefore holds for any non-negative vectors: an unnormalised probability row and a stale sample buffer were both injected into the harness and both passed at 10^{-15} while the metrics printed beside them were wrong. What it tests is the fold arithmetic and not the probabilities that entered it. A cheaper selftest reaches the same arithmetic first, at 10^{-12}, but only at $V = 2$ and $S \leq 2$.

7 Limitations

The scale bounds the milliseconds, not the counts. 11,061,531 parameters on a 6 GB card, picked so all four arms fit at once. Touch counts do not depend on P, and neither does the flat numerator of Section 5.1, but every millisecond and every GB/s here belongs to this card. Where the network becomes token-bound does depend on the model: a larger one saturates the card at a smaller B. The vocabulary of 27 keeps the token embedding negligible, so nothing covers a posterior

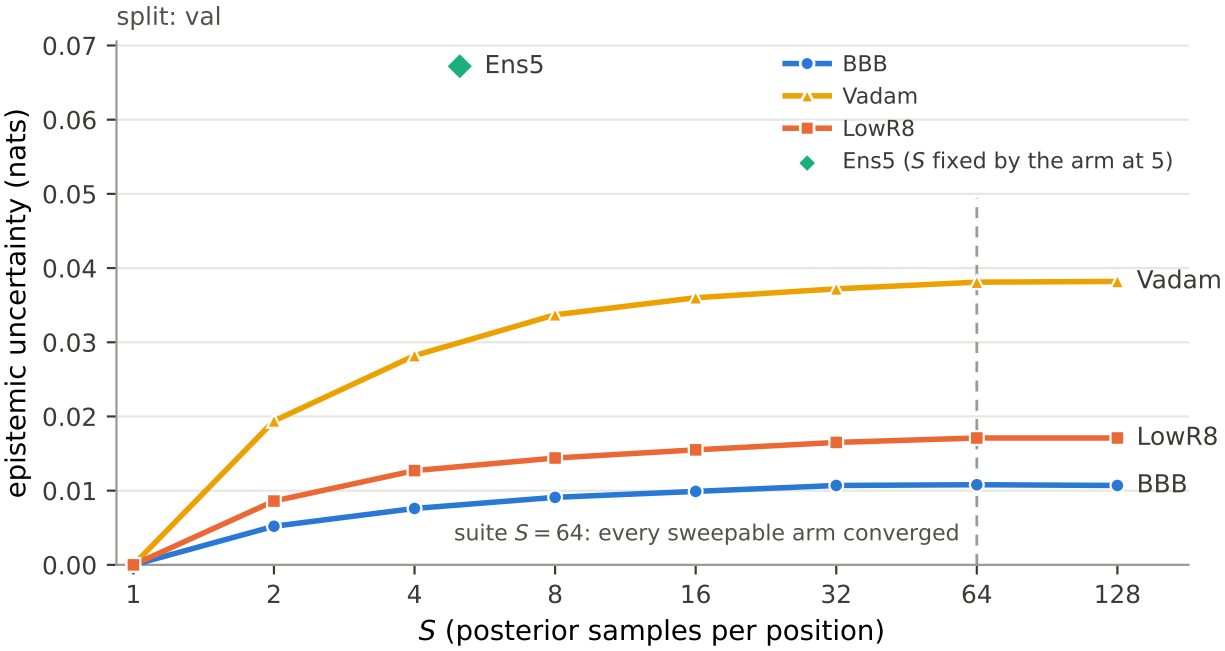


Figure 1: Epistemic uncertainty against posterior samples S, validation split. The foot at $S = 1$ is exactly zero by construction; the ensemble is a marked point at the members it was paid to train.

over a large embedding table.

The measurement is bounded by the card and by one axis. The card throttles, so the two sweeps are comparable only to about 1%: with per-point cooldown the later one still started a few degrees warmer. That is what the unchanged control arm is for, it is where the band of Section 5.2 comes from, and no claim in that section rests on a difference smaller than it. The low-rank arm at $B = 256$ allocated with 248 MB of 6144 MB free and then ran at about four times what its own curve predicts, because past a couple of hundred megabytes of headroom the driver backs allocations with host memory; on WSL2 `cudaMalloc` succeeds in that state, so that arm's sweep ends at $B = 128$. The sweep varies B at $T = 64$ throughout, so what is measured is a dependence on batch size and not on tokens per step. One training run per arm, no seed sweep, no error bars, and three backward kernels accumulate with `atomicAdd`, so training is not bit-reproducible in principle; normal-mode runs at this configuration do reproduce, which is what the trajectory check of Section 6 uses, and one-batch overfit runs do not.

The quality comparison is confounded and unconverged. 100,000 steps at 256 tokens per step is 28% of one epoch, so every quality number is what a method does at a fixed step budget. Loss tracks the injected noise scale monotonically across arms, so an arm that samples less aggressively looks better on predictive metrics for a reason unrelated to its posterior, and the off-distribution stream is uniform-random ids rather than shifted text, supporting no comparison with the shift literature [19]. The low-rank arm is Bayesian over 96% of the model and the ensemble evaluates at $S = E = 5$, so a mixture metric read across arms compares different coverage at different sample counts.

8 Adding an arm

A method is added by writing two functions in one file. The update kernel takes what the method stores and the gradient the shared backward pass produced, and writes the state and the next step's draw. The `uq_sample_fn` takes a sample index and returns a device pointer to draw s. The rest is inherited.

Three conventions the shared code depends on, each of which fails silently. An arm's `main` is compiled out when its file is included into the Python extension, because four of them cannot coexist in one module, while its kernels and its callback stay outside that guard so the module gets the arm's mathematics with no second copy. cuBLAS is reached only through an accessor macro, which yields one handle per linked image whatever the translation-unit layout. And a fatal path goes through `UNCERTAIN_ABORT`, `exit` in a binary and a thrown exception in the module, so shared code cannot kill an interpreter.

A new arm is then measured by the commands the four here were: `bench` at one batch size for a point on the sweep, `bench_sweep.sh` for the whole of it, and `eval` for the quality and uncertainty columns at a stated S and split. Tables 2 and 6 are the baseline it lands beside. `docs/adding-an-arm.md` in the repository is the long form: which file to copy, how the buffer layout is extended, how to count the passes a step makes, and what to run before a number from a new arm is reported.

References

[1] Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural network. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 1613–1622. PMLR, 2015. URL https://proceedings.mlr.press/v37/blundell15.html.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

[3] Erik A. Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux — effortless Bayesian deep learning. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, pages 20089–20103, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/a7c9585703d275249f30a088cebba0ad-Abstract.html.

[4] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1050–1059. JMLR.org, 2016. URL http://proceedings.mlr.press/v48/gal16.html.

[5] Alex Graves. Practical variational inference for neural networks. In *Advances in Neural Information Processing Systems 24 (NIPS)*, pages 2348–2356, 2011. URL https://proceedings.neurips.cc/paper/2011/hash/7eb3c8be3d411e8ebfab08eba5f49632-Abstract.html.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*,

volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

[7] Geoffrey E. Hinton and Drew van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the Sixth Annual ACM Conference on Computational Learning Theory (COLT)*, pages 5–13. ACM, 1993. doi: 10.1145/168304.168306.

[8] Andrej Karpathy. nanoGPT. https://github.com/karpathy/nanoGPT, 2022.

[9] Andrej Karpathy. llm.c: LLM training in simple, raw C/CUDA. https://github.com/karpathy/llm.c, 2024.

[10] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 5574–5584, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/2650d6089a6d640c5e85b2b88265dc2b-Abstract.html.

[11] Mohammad Emtiyaz Khan, Didrik Nielsen, Voot Tangkaratt, Wu Lin, Yarin Gal, and Akash Srivastava. Fast and scalable Bayesian deep learning by weight-perturbation in Adam. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 2611–2620. PMLR, 2018. URL https://proceedings.mlr.press/v80/khan18a.html.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL http://arxiv.org/abs/1412.6980.

[13] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *2nd International Conference on Learning Representations (ICLR)*, 2014. URL http://arxiv.org/abs/1312.6114.

[14] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 6402–6413, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html.

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

[16] Wesley J. Maddox, Pavel Izmailov, Timur Garipov, Dmitry P. Vetrov, and Andrew Gordon Wilson. A simple baseline for Bayesian uncertainty in deep learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13132–13143, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/118921efba23fc329e6560b27861f0c2-Abstract.html.

[17] Matt Mahoney. About the test data. http://mattmahoney.net/dc/textdata.html, 2011. Large Text Compression Benchmark; source of `enwik8` and `text8`.

[18] Kazuki Osawa, Siddharth Swaroop, Mohammad Emtiyaz Khan, Anirudh Jain, Runa Eschenhagen, Richard E. Turner, and Rio Yokota. Practical deep learning with Bayesian principles. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 4289–4301, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/b53477c2821c1bf0da5d40e57b870d35-Abstract.html.

[19] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua V. Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13969–13980, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html.

[20] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[21] Hippolyt Ritter, Aleksandar Botev, and David Barber. A scalable Laplace approximation for neural networks. In *6th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2018. URL https://openreview.net/forum?id=Skdvd2xAZ.

[22] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pages 16:1–16:12. ACM, 2011. doi: 10.1145/2063384.2063405.

[23] Yuesong Shen, Nico Daheim, Bai Cong, Peter Nickl, Gian Maria Marconi, Clement Bazan, Rio Yokota, Iryna Gurevych, Daniel Cremers, Mohammad Emtiyaz Khan, and Thomas Möllenhoff. Variational learning is effective for large deep networks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 44665–44686. PMLR, 2024. URL https://proceedings.mlr.press/v235/shen24b.html.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 5998–6008, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[25] Yeming Wen, Paul Vicol, Jimmy Ba, Dustin Tran, and Roger B. Grosse. Flipout: Efficient pseudo-independent weight perturbations on mini-batches. In *6th International Conference on Learning Representations (ICLR)*. OpenReview.net, 2018. URL https://openreview.net/forum?id=rJNpifWAb.